Valentin LE GALL
Elia LEPREVOST

Bases de données réparties

Nano-Satellites

PHASE 4
Exploitation avancée & Optimisation
Projet

ALTN83-2526BSP01 - Bases de données réparties
Christophe CROISANT

1) Script Vues **3**
V1 — v_satellites_operationnels 3
V2 — v_fenetres_detail 3
V3 — v_stats_missions 5
V4 — mv_volumes_mensuels 5
2) Script CTE & Sous-requêtes **6**
Exercice 5 — CTE simple 6
Exercice 6 — CTE multiples 7
Exercice 7 — CTE récursive 9
Exercice 8 — Sous-requête scalaire 12
Exercice 9 — Sous-requête corrélée 13
Exercice 10 — EXISTS / NOT EXISTS 13
3) Script Analytiques & MERGE **15**
Exercice 11 — ROW_NUMBER, RANK, DENSE_RANK 15
Exercice 12 — LAG / LEAD 16
Exercice 13 — SUM OVER 17
Exercice 14 — Tableau de bord constellation 18
Exercice 15 — Synchronisation de statuts satellites 20
Exercice 16 — Synchronisation des affectations de stations 21
4) Script Index & EXPLAIN PLAN **23**
Exercice 17 — Créer les index stratégiques manquants 23
Exercice 18 — Lire et interpréter un EXPLAIN PLAN 25
Exercice 19 — Mesurer l’impact d’un index invisible 31
Rapport de pilotage intégral — Requête de synthèse finale **34**

1) Script Vues

V1 — v_satellites_operationnels

```
CREATE OR REPLACE VIEW v_satellites_operationnels AS
SELECT
    s.id_satellite,
    s.nom_satellite,
    s.format_cubesat,
    s.capacite_batterie,
    s.statut,
    o.type_orbite,
    o.altitude,
    COUNT(e.ref_instrument) AS nb_instruments
FROM SATELLITE s
         JOIN ORBITE o
              ON s.id_orbite = o.id_orbite
         LEFT JOIN EMBARQUEMENT e
                   ON s.id_satellite = e.id_satellite
WHERE s.statut = 'Opérationnel'
GROUP BY
    s.id_satellite,
    s.nom_satellite,
    s.format_cubesat,
    s.capacite_batterie,
    s.statut,
    o.type_orbite,
    o.altitude;
```

Test :

```
SELECT * FROM v_satellites_operationnels;
```

Résultat :

```
SAT-001,NanoOrbit-Alpha,3U,20.0,Opérationnel,SSO,550,2
SAT-002,NanoOrbit-Beta,3U,20.0,Opérationnel,SSO,550,1
SAT-003,NanoOrbit-Gamma,6U,40.0,Opérationnel,SSO,700,2
```

V2 — v_fenetres_detail

```
CREATE OR REPLACE VIEW v_fenetres_detail AS
SELECT
```

```
    f.id_fenetre,
    f.datetime_debut,
    f.duree,
    TO_CHAR(NUMTODSINTERVAL(f.duree, 'SECOND'), 'HH24:MI:SS') AS
duree_formatee,
    f.volume_donnees,
    f.statut,
    s.nom_satellite,
    st.nom_station,
    c.nom_centre
FROM FENETRE_COM f
JOIN SATELLITE s
    ON f.id_satellite = s.id_satellite
JOIN STATION_SOL st
    ON f.code_station = st.code_station
LEFT JOIN AFFECTATION_STATION af
    ON st.code_station = af.code_station
LEFT JOIN CENTRE_CONTROLE c
    ON af.id_centre = c.id_centre;
```

Test :

```
SELECT * FROM v_fenetres_detail;
```

Résultat :

```
1,2024-01-15 09:14:00.000000,420,+000000000
00:07:00.000000000,1250.0,Réalisée,NanoOrbit-Alpha,Kiruna Arctic
Station,NanoOrbit Paris HQ
4,2024-01-20 14:22:00.000000,380,+000000000
00:06:20.000000000,,Planifiée,NanoOrbit-Alpha,Toulouse Ground
Station,NanoOrbit Paris HQ
2,2024-01-15 11:52:00.000000,310,+000000000
00:05:10.000000000,890.0,Réalisée,NanoOrbit-Beta,Toulouse Ground
Station,NanoOrbit Paris HQ
3,2024-01-16 08:30:00.000000,540,+000000000
00:09:00.000000000,1680.0,Réalisée,NanoOrbit-Gamma,Kiruna Arctic
Station,NanoOrbit Paris HQ
5,2024-01-21 07:45:00.000000,290,+000000000
00:04:50.000000000,,Planifiée,NanoOrbit-Gamma,Toulouse Ground
Station,NanoOrbit Paris HQ
```

V3 — v_stats_missions

```
CREATE OR REPLACE VIEW v_stats_missions AS
SELECT
    m.id_mission,
    m.nom_mission,
    COUNT(DISTINCT p.id_satellite) AS nb_satellites,
    COUNT(DISTINCT o.type_orbite) AS nb_types_orbite,
    NVL(SUM(f.volume_donnees), 0) AS volume_total
FROM MISSION m
LEFT JOIN PARTICIPATION p
    ON m.id_mission = p.id_mission
LEFT JOIN SATELLITE s
    ON p.id_satellite = s.id_satellite
LEFT JOIN ORBITE o
    ON s.id_orbite = o.id_orbite
LEFT JOIN FENETRE_COM f
    ON s.id_satellite = f.id_satellite
GROUP BY
    m.id_mission,
    m.nom_mission;
```

Test :

```
SELECT * FROM v_stats_missions;
```

Résultat :

```
MSN-ARC-2023,ArcticWatch 2023,3,1,3820
MSN-DEF-2022,DeforestAlert,2,2,1250
MSN-COAST-2024,CoastGuard 2024,2,1,1680
```

V4 — mv_volumes_mensuels

```
CREATE MATERIALIZED VIEW mv_volumes_mensuels
BUILD IMMEDIATE
REFRESH ON DEMAND
AS
SELECT
    TRUNC(f.datetime_debut, 'MM') AS mois,
    c.nom_centre,
    s.format_cubesat,
```

```
    SUM(NVL(f.volume_donnees, 0)) AS volume_total
FROM FENETRE_COM f
JOIN SATELLITE s
    ON f.id_satellite = s.id_satellite
JOIN STATION_SOL st
    ON f.code_station = st.code_station
JOIN AFFECTATION_STATION af
    ON st.code_station = af.code_station
JOIN CENTRE_CONTROLE c
    ON af.id_centre = c.id_centre
WHERE f.statut = 'Réalisée'
GROUP BY
    TRUNC(f.datetime_debut, 'MM'),
    c.nom_centre,
    s.format_cubesat;
```

Refresh manuel :

```
EXEC DBMS_MVIEW.REFRESH('MV_VOLUMES_MENSUELS');
```

Test :

```
SELECT * FROM mv_volumes_mensuels;
```

Résultat :

```
2024-01-01,NanoOrbit Paris HQ,3U,2140
2024-01-01,NanoOrbit Paris HQ,6U,1680
```

2) Script CTE & Sous-requêtes

Exercice 5 — CTE simple

```
WITH stats_satellites AS (
    SELECT
        s.id_satellite,
        s.nom_satellite,
        COUNT(f.id_fenetre) AS nb_fenetres_realisees,
        NVL(SUM(f.volume_donnees), 0) AS volume_total,
        ROUND(NVL(AVG(f.volume_donnees), 0), 2) AS volume_moyen
    FROM SATELLITE s
    LEFT JOIN FENETRE_COM f
        ON s.id_satellite = f.id_satellite
       AND f.statut = 'Réalisée'
```

```
    GROUP BY
        s.id_satellite,
        s.nom_satellite
)
SELECT *
FROM (
    SELECT
        id_satellite,
        nom_satellite,
        nb_fenetres_realisees,
        volume_total,
        volume_moyen
    FROM stats_satellites
    ORDER BY volume_total DESC
)
WHERE ROWNUM <= 3;
```

Résultat :

```
SAT-003,NanoOrbit-Gamma,1,1680,1680
SAT-001,NanoOrbit-Alpha,1,1250,1250
SAT-002,NanoOrbit-Beta,1,890,890
```

Analyse :

Cette requête calcule d'abord les statistiques par satellite dans une CTE, puis extrait les 3 meilleurs volumes globaux. Avec le jeu de données de référence, les fenêtres réalisées sont les lignes 1, 2 et 3, avec respectivement des volumes 1250, 890 et 1680, ce qui rend cet exercice particulièrement pertinent pour classer SAT-003, SAT-001 et SAT-002.

Exercice 6 — CTE multiples

```
WITH fenetres_mois AS (
    SELECT
        c.id_centre,
        c.nom_centre,
        st.code_station,
        st.nom_station,
        f.id_fenetre,
        f.volume_donnees
    FROM CENTRE_CONTROLE c
    JOIN AFFECTATION_STATION af
        ON c.id_centre = af.id_centre
```

```sql
    JOIN STATION_SOL st
        ON af.code_station = st.code_station
    LEFT JOIN FENETRE_COM f
        ON st.code_station = f.code_station
       AND TRUNC(f.datetime_debut, 'MM') = TRUNC(SYSDATE, 'MM')
),
stats_centre AS (
    SELECT
        id_centre,
        nom_centre,
        COUNT(id_fenetre) AS nb_fenetres_mois,
        NVL(SUM(volume_donnees), 0) AS volume_total_mois
    FROM fenetres_mois
    GROUP BY
        id_centre,
        nom_centre
),
stats_station AS (
    SELECT
        id_centre,
        nom_centre,
        code_station,
        nom_station,
        COUNT(id_fenetre) AS nb_fenetres_station,
        ROW_NUMBER() OVER (
            PARTITION BY id_centre
            ORDER BY COUNT(id_fenetre) DESC, code_station
        ) AS rn
    FROM fenetres_mois
    GROUP BY
        id_centre,
        nom_centre,
        code_station,
        nom_station
)
SELECT
    sc.id_centre,
    sc.nom_centre,
    sc.nb_fenetres_mois,
    sc.volume_total_mois,
    ss.code_station AS station_plus_active,
    ss.nom_station
FROM stats_centre sc
LEFT JOIN stats_station ss
```

```
    ON sc.id_centre = ss.id_centre
   AND ss.rn = 1
ORDER BY sc.id_centre;
```

Résultat :

```
CTR-001,NanoOrbit Paris HQ,0,0,GS-KIR-01,Kiruna Arctic Station
CTR-002,NanoOrbit Houston,0,0,GS-SGP-01,Singapore Station
```

Analyse

Cette solution sépare proprement les calculs en plusieurs étapes :

- une première CTE pour récupérer les fenêtres du mois par centre,
- une deuxième pour agréger les indicateurs du centre,
- une troisième pour identifier la station la plus active.

Dans NanoOrbit, CTR-001 supervise GS-TLS-01 et GS-KIR-01, tandis que CTR-002 supervise GS-SGP-01. Le jeu de données initial ne contient que ces affectations-là.

Dans le résultat de cette requête, nous n'avons aucune fenêtre et donc aucun volume car, dans le jeu de données initial, les fenêtres sont à janvier 2024. En adaptant la requête à ce mois, nous avons ceci :

```
CTR-001,NanoOrbit Paris HQ,5,3820,GS-TLS-01,Toulouse Ground Station
CTR-002,NanoOrbit Houston,0,0,GS-SGP-01,Singapore Station
```

Exercice 7 — CTE récursive

```
WITH liens AS (
    -- Liens CENTRE -> STATION
    SELECT
        'CENTRE:' || c.id_centre AS parent_id,
        'STATION:' || st.code_station AS node_id,
        2 AS niveau,
        'STATION' AS type_element,
        st.nom_station AS libelle,
        c.id_centre,
        st.code_station,
        CAST(NULL AS NUMBER) AS id_fenetre,
        CAST(NULL AS TIMESTAMP) AS datetime_debut
    FROM CENTRE_CONTROLE c
             JOIN AFFECTATION_STATION af
                  ON af.id_centre = c.id_centre
```

```sql
            JOIN STATION_SOL st
                 ON st.code_station = af.code_station

    UNION ALL

    -- Liens STATION -> FENETRE
    SELECT
        'STATION:' || f.code_station AS parent_id,
        'FENETRE:' || TO_CHAR(f.id_fenetre) AS node_id,
        3 AS niveau,
        'FENETRE' AS type_element,
        'Fenêtre ' || f.id_fenetre || ' - ' ||
        TO_CHAR(f.datetime_debut, 'YYYY-MM-DD HH24:MI') AS libelle,
        af.id_centre,
        f.code_station,
        f.id_fenetre,
        f.datetime_debut
    FROM FENETRE_COM f
            JOIN AFFECTATION_STATION af
                 ON af.code_station = f.code_station
    WHERE f.datetime_debut >= TIMESTAMP '2024-01-01 00:00:00'
),
     hierarchie (
                niveau,
                type_element,
                libelle,
                node_id,
                parent_id,
                id_centre,
                code_station,
                id_fenetre,
                datetime_debut
        ) AS (
        -- Ancrage : centres
        SELECT
            1 AS niveau,
            'CENTRE' AS type_element,
            c.nom_centre AS libelle,
            'CENTRE:' || c.id_centre AS node_id,
            CAST(NULL AS VARCHAR2(50)) AS parent_id,
            c.id_centre,
            CAST(NULL AS VARCHAR2(20)) AS code_station,
            CAST(NULL AS NUMBER) AS id_fenetre,
            CAST(NULL AS TIMESTAMP) AS datetime_debut
```

```sql
        FROM CENTRE_CONTROLE c

        UNION ALL

        SELECT
            l.niveau,
            l.type_element,
            l.libelle,
            l.node_id,
            l.parent_id,
            l.id_centre,
            l.code_station,
            l.id_fenetre,
            l.datetime_debut
        FROM liens l
                 JOIN hierarchie h
                      ON l.parent_id = h.node_id
     )
SELECT
    LPAD(' ', (niveau - 1) * 4) || libelle AS arborescence,
    type_element,
    id_centre,
    code_station,
    id_fenetre,
    datetime_debut
FROM hierarchie
ORDER BY
    id_centre,
    niveau,
    code_station NULLS FIRST,
    datetime_debut NULLS FIRST;
```

Résultat :

```
NanoOrbit Paris HQ,CENTRE,CTR-001,,,
    Kiruna Arctic Station,STATION,CTR-001,GS-KIR-01,,
    Toulouse Ground Station,STATION,CTR-001,GS-TLS-01,,
        Fenêtre 1 - 2024-01-15
09:14,FENETRE,CTR-001,GS-KIR-01,1,2024-01-15 09:14:00.000000
        Fenêtre 3 - 2024-01-16
08:30,FENETRE,CTR-001,GS-KIR-01,3,2024-01-16 08:30:00.000000
        Fenêtre 2 - 2024-01-15
11:52,FENETRE,CTR-001,GS-TLS-01,2,2024-01-15 11:52:00.000000
        Fenêtre 4 - 2024-01-20
```

```
14:22,FENETRE,CTR-001,GS-TLS-01,4,2024-01-20 14:22:00.000000
        Fenêtre 5 - 2024-01-21
07:45,FENETRE,CTR-001,GS-TLS-01,5,2024-01-21 07:45:00.000000
NanoOrbit Houston,CENTRE,CTR-002,,,
    Singapore Station,STATION,CTR-002,GS-SGP-01,,
```

Exercice 8 — Sous-requête scalaire

```sql
SELECT
    f.id_fenetre,
    f.datetime_debut,
    f.volume_donnees,
    ROUND(
        f.volume_donnees - (
            SELECT AVG(f2.volume_donnees)
            FROM FENETRE_COM f2
            WHERE f2.volume_donnees IS NOT NULL
        ),
        2
    ) AS ecart_moyenne
FROM FENETRE_COM f
WHERE f.volume_donnees IS NOT NULL
  AND f.volume_donnees > (
      SELECT AVG(f2.volume_donnees)
      FROM FENETRE_COM f2
      WHERE f2.volume_donnees IS NOT NULL
  )
ORDER BY f.volume_donnees DESC;
```

Résultat :

```
3,2024-01-16 08:30:00.000000,1680.0,406.67
```

Analyse

La sous-requête calcule la moyenne générale des volumes réels, puis la requête principale filtre les fenêtres au-dessus de cette moyenne. Dans NanoOrbit, seules les fenêtres au statut Réalisée possèdent un volume_donnees, alors que les fenêtres Planifiée doivent rester à NULL. C'est cohérent avec la règle métier RG-F05 et avec les données du référentiel.

Exercice 9 — Sous-requête corrélée

```
SELECT
    s.id_satellite,
    s.nom_satellite,
    f.datetime_debut AS derniere_fenetre,
    st.nom_station,
    f.volume_donnees
FROM SATELLITE s
JOIN FENETRE_COM f
    ON f.id_satellite = s.id_satellite
JOIN STATION_SOL st
    ON st.code_station = f.code_station
WHERE f.statut = 'Réalisée'
  AND f.datetime_debut = (
      SELECT MAX(f2.datetime_debut)
      FROM FENETRE_COM f2
      WHERE f2.id_satellite = s.id_satellite
        AND f2.statut = 'Réalisée'
  )
ORDER BY s.id_satellite;
```

Résultat :

```
SAT-001,NanoOrbit-Alpha,2024-01-15 09:14:00.000000,Kiruna Arctic
Station,1250.0
SAT-002,NanoOrbit-Beta,2024-01-15 11:52:00.000000,Toulouse Ground
Station,890.0
SAT-003,NanoOrbit-Gamma,2024-01-16 08:30:00.000000,Kiruna Arctic
Station,1680.0
```

Analyse

La sous-requête est corrélée parce qu'elle dépend du satellite courant. Elle retourne la date maximale de fenêtre réalisée pour chaque satellite. Avec le jeu de données initial, on retrouve notamment des fenêtres réalisées pour SAT-001, SAT-002 et SAT-003.

Exercice 10 — EXISTS / NOT EXISTS

10.A — Satellites sans aucune fenêtre réalisée

```
SELECT
    s.id_satellite,
    s.nom_satellite,
    s.statut
```

```sql
FROM SATELLITE s
WHERE NOT EXISTS (
    SELECT 1
    FROM FENETRE_COM f
    WHERE f.id_satellite = s.id_satellite
      AND f.statut = 'Réalisée'
)
ORDER BY s.id_satellite;
```

Résultat :

```
SAT-004,NanoOrbit-Delta,En veille
SAT-005,NanoOrbit-Epsilon,Désorbité
```

10.B — Stations n'ayant traité aucune fenêtre ce trimestre

```sql
SELECT
    st.code_station,
    st.nom_station,
    st.statut
FROM STATION_SOL st
WHERE NOT EXISTS (
    SELECT 1
    FROM FENETRE_COM f
    WHERE f.code_station = st.code_station
      AND f.datetime_debut >= DATE '2024-01-01'
      AND f.datetime_debut <  DATE '2024-04-01'
)
ORDER BY st.code_station;
```

Résultat :

```
GS-SGP-01,Singapore Station,Maintenance
```

Commentaire d'analyse

Une station peut n'avoir traité aucune fenêtre sur une période donnée pour plusieurs raisons :

- maintenance,
- absence de planification,
- faible sollicitation opérationnelle.

Dans le jeu NanoOrbit, GS-SGP-01 est précisément en statut Maintenance, ce qui en fait un très bon cas métier pour cet exercice.

3) Script Analytiques & MERGE

Exercice 11 — ROW_NUMBER, RANK, DENSE_RANK

```sql
WITH volumes_sat AS (
    SELECT
        s.id_satellite,
        s.nom_satellite,
        o.type_orbite,
        NVL(SUM(f.volume_donnees), 0) AS volume_total
    FROM SATELLITE s
    JOIN ORBITE o
        ON s.id_orbite = o.id_orbite
    LEFT JOIN FENETRE_COM f
        ON f.id_satellite = s.id_satellite
       AND f.statut = 'Réalisée'
    GROUP BY
        s.id_satellite,
        s.nom_satellite,
        o.type_orbite
)
SELECT
    id_satellite,
    nom_satellite,
    type_orbite,
    volume_total,
    ROW_NUMBER() OVER (ORDER BY volume_total DESC) AS rn_global,
    RANK() OVER (ORDER BY volume_total DESC) AS rank_global,
    DENSE_RANK() OVER (ORDER BY volume_total DESC) AS dense_rank_global,
    ROW_NUMBER() OVER (
        PARTITION BY type_orbite
        ORDER BY volume_total DESC
    ) AS rn_par_orbite,
    RANK() OVER (
        PARTITION BY type_orbite
        ORDER BY volume_total DESC
    ) AS rank_par_orbite,
    DENSE_RANK() OVER (
        PARTITION BY type_orbite
        ORDER BY volume_total DESC
    ) AS dense_rank_par_orbite
FROM volumes_sat
ORDER BY volume_total DESC, id_satellite;
```

Résultat :

```
SAT-003,NanoOrbit-Gamma,SSO,1680,1,1,1,1,1,1
SAT-001,NanoOrbit-Alpha,SSO,1250,2,2,2,2,2,2
SAT-002,NanoOrbit-Beta,SSO,890,3,3,3,3,3,3
SAT-004,NanoOrbit-Delta,SSO,0,4,4,4,4,4,4
SAT-005,NanoOrbit-Epsilon,LEO,0,5,4,4,1,1,1
```

Analyse

Cette requête calcule le volume total téléchargé par satellite à partir des fenêtres Réalisée, puis compare plusieurs types de classement :

- ROW_NUMBER : rang strict,
- RANK : ex æquo avec trous,
- DENSE_RANK : ex æquo sans trous.

Le jeu de données de référence contient des volumes réalisés sur SAT-001, SAT-002 et SAT-003, ce qui donne une base cohérente pour le classement.

Exercice 12 — LAG / LEAD

```
SELECT
    f.code_station,
    st.nom_station,
    f.id_fenetre,
    f.datetime_debut,
    f.volume_donnees,
    LAG(f.volume_donnees) OVER (
        PARTITION BY f.code_station
        ORDER BY f.datetime_debut
    ) AS volume_precedent,
    LEAD(f.volume_donnees) OVER (
        PARTITION BY f.code_station
        ORDER BY f.datetime_debut
    ) AS volume_suivant,
    ROUND(
        CASE
            WHEN LAG(f.volume_donnees) OVER (
                PARTITION BY f.code_station
                ORDER BY f.datetime_debut
            ) IS NULL
             OR LAG(f.volume_donnees) OVER (
                PARTITION BY f.code_station
                ORDER BY f.datetime_debut
            ) = 0
```

```
            THEN NULL
            ELSE (
                (f.volume_donnees -
                 LAG(f.volume_donnees) OVER (
                    PARTITION BY f.code_station
                    ORDER BY f.datetime_debut
                 ))
                / LAG(f.volume_donnees) OVER (
                    PARTITION BY f.code_station
                    ORDER BY f.datetime_debut
                  )
            ) * 100
        END,
        2
    ) AS evolution_pct
FROM FENETRE_COM f
JOIN STATION_SOL st
    ON st.code_station = f.code_station
WHERE f.statut = 'Réalisée'
ORDER BY f.code_station, f.datetime_debut;
```

Résultat :

```
GS-KIR-01,Kiruna Arctic Station,1,2024-01-15
09:14:00.000000,1250.0,,1680,
GS-KIR-01,Kiruna Arctic Station,3,2024-01-16
08:30:00.000000,1680.0,1250,,34.4
GS-TLS-01,Toulouse Ground Station,2,2024-01-15 11:52:00.000000,890.0,,,
```

Analyse

LAG permet de récupérer le volume de la fenêtre précédente pour la même station, tandis que LEAD permet de regarder la suivante. Le pourcentage d'évolution est très utile dans un contexte de monitoring.
Dans les données NanoOrbit, GS-KIR-01 a plusieurs fenêtres réalisées, ce qui rend cet exercice particulièrement intéressant.

Exercice 13 — SUM OVER

```
SELECT
    c.id_centre,
    c.nom_centre,
    f.id_fenetre,
    f.datetime_debut,
```

```
    f.volume_donnees,
    SUM(f.volume_donnees) OVER (
        PARTITION BY c.id_centre
        ORDER BY f.datetime_debut
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS volume_cumule,
    ROUND(
        AVG(f.volume_donnees) OVER (
            PARTITION BY c.id_centre
            ORDER BY f.datetime_debut
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
        ),
        2
    ) AS moyenne_mobile_3
FROM FENETRE_COM f
JOIN STATION_SOL st
    ON st.code_station = f.code_station
JOIN AFFECTATION_STATION af
    ON af.code_station = st.code_station
JOIN CENTRE_CONTROLE c
    ON c.id_centre = af.id_centre
WHERE f.statut = 'Réalisée'
ORDER BY c.id_centre, f.datetime_debut;
```

Résultat :

```
CTR-001,NanoOrbit Paris HQ,1,2024-01-15 09:14:00.000000,1250.0,1250,1250
CTR-001,NanoOrbit Paris HQ,2,2024-01-15 11:52:00.000000,890.0,2140,1070
CTR-001,NanoOrbit Paris HQ,3,2024-01-16
08:30:00.000000,1680.0,3820,1273.33
```

Analyse

Cette requête produit deux indicateurs de suivi :

- le volume cumulé dans le temps pour chaque centre,
- la moyenne mobile sur 3 fenêtres, utile pour lisser l'activité récente.

Comme CTR-001 supervise GS-TLS-01 et GS-KIR-01, c'est surtout lui qui ressort dans le jeu initial avec les fenêtres réalisées.

Exercice 14 — Tableau de bord constellation

```
WITH volumes_mensuels AS (
    SELECT
        s.id_satellite,
```

```sql
        s.nom_satellite,
        NVL(SUM(f.volume_donnees), 0) AS volume_mois
    FROM SATELLITE s
    LEFT JOIN FENETRE_COM f
        ON f.id_satellite = s.id_satellite
       AND f.statut = 'Réalisée'
       AND f.datetime_debut >= DATE '2024-01-01'
       AND f.datetime_debut <  DATE '2024-02-01'
    GROUP BY
        s.id_satellite,
        s.nom_satellite
)
SELECT
    id_satellite,
    nom_satellite,
    volume_mois,
    RANK() OVER (ORDER BY volume_mois DESC) AS rang_satellite,
    ROUND(
        volume_mois * 100 /
        NULLIF(SUM(volume_mois) OVER (), 0),
        2
    ) AS part_volume_pct,
    SUM(volume_mois) OVER (
        ORDER BY volume_mois DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS cumul_volume,
    ROUND(AVG(volume_mois) OVER (), 2) AS moyenne_generale,
    ROUND(volume_mois - AVG(volume_mois) OVER (), 2) AS ecart_moyenne
FROM volumes_mensuels
ORDER BY volume_mois DESC, id_satellite;
```

Résultat :

```
SAT-003,NanoOrbit-Gamma,1680,1,43.98,1680,764,916
SAT-001,NanoOrbit-Alpha,1250,2,32.72,2930,764,486
SAT-002,NanoOrbit-Beta,890,3,23.3,3820,764,126
SAT-004,NanoOrbit-Delta,0,4,0,3820,764,-764
SAT-005,NanoOrbit-Epsilon,0,4,0,3820,764,-764
```

Analyse

Cette requête fournit un mini tableau de bord :

- rang du satellite,
- part du volume global,

- cumul progressif,
- comparaison à la moyenne.

Comme les fenêtres de référence sont concentrées en janvier 2024, figer le mois permet d'éviter des résultats vides.

Exercice 15 — Synchronisation de statuts satellites

```
MERGE INTO SATELLITE s
USING (
    SELECT 'SAT-003' AS id_satellite, 'Défaillant' AS statut, 'ORB-002'
AS id_orbite FROM dual
    UNION ALL
    SELECT 'SAT-004', 'Opérationnel', 'ORB-002' FROM dual
    UNION ALL
    SELECT 'SAT-006', 'En veille', 'ORB-003' FROM dual
) src
ON (s.id_satellite = src.id_satellite)
WHEN MATCHED THEN
    UPDATE SET
                s.statut = src.statut,
                s.id_orbite = src.id_orbite
WHEN NOT MATCHED THEN
    INSERT (
        id_satellite,
        nom_satellite,
        date_lancement,
        masse,
        format_cubesat,
        statut,
        duree_vie_prevue,
        capacite_batterie,
        id_orbite
    )
    VALUES (
                src.id_satellite,
                'Nouveau satellite ' || src.id_satellite,
                DATE '2026-01-01',
                1.00,
                '3U',
                'En veille',
                36,
                15,
                src.id_orbite
            );
```

Test :

```
SELECT id_satellite, nom_satellite, statut, id_orbite
FROM SATELLITE
WHERE id_satellite IN ('SAT-003', 'SAT-004', 'SAT-006')
ORDER BY id_satellite;
```

Résultat :

```
SAT-003,NanoOrbit-Gamma,Défaillant,ORB-002
SAT-004,NanoOrbit-Delta,Opérationnel,ORB-002
SAT-006,Nouveau satellite SAT-006,En veille,ORB-003
```

Exercice 16 — Synchronisation des affectations de stations

Étape 1 — créer le centre si nécessaire

```
MERGE INTO CENTRE_CONTROLE c
USING (
    SELECT
        'CTR-003' AS id_centre,
        'NanoOrbit Singapore' AS nom_centre,
        'Singapour' AS ville,
        'Asie-Pacifique' AS region_geo,
        'Asia/Singapore' AS fuseau_horaire,
        'Actif' AS statut
    FROM dual
) src
ON (c.id_centre = src.id_centre)
WHEN MATCHED THEN
    UPDATE SET
        c.nom_centre = src.nom_centre,
        c.ville = src.ville,
        c.region_geo = src.region_geo,
        c.fuseau_horaire = src.fuseau_horaire,
        c.statut = src.statut
WHEN NOT MATCHED THEN
    INSERT (
        id_centre,
        nom_centre,
        ville,
        region_geo,
        fuseau_horaire,
```

```
        statut
    )
    VALUES (
        src.id_centre,
        src.nom_centre,
        src.ville,
        src.region_geo,
        src.fuseau_horaire,
        src.statut
    );
```

Étape 2 — synchroniser les affectations

```
MERGE INTO AFFECTATION_STATION a
USING (
    SELECT 'CTR-001' AS id_centre, 'GS-TLS-01' AS code_station, DATE
'2024-01-01' AS date_affectation FROM dual
    UNION ALL
    SELECT 'CTR-001', 'GS-KIR-01', DATE '2024-01-01' FROM dual
    UNION ALL
    SELECT 'CTR-003', 'GS-SGP-01', DATE '2024-02-01' FROM dual
) src
ON (
    a.id_centre = src.id_centre
    AND a.code_station = src.code_station
)
WHEN MATCHED THEN
    UPDATE SET
        a.date_affectation = src.date_affectation
WHEN NOT MATCHED THEN
    INSERT (
        id_centre,
        code_station,
        date_affectation,
        commentaire
    )
    VALUES (
        src.id_centre,
        src.code_station,
        src.date_affectation,
        'Synchronisation depuis configuration révisée'
    );
```

Test :

```sql
SELECT id_centre, code_station, date_affectation, commentaire
FROM AFFECTATION_STATION
ORDER BY id_centre, code_station;
```

Résultat :

```
CTR-001,GS-KIR-01,2024-01-01,Paris HQ supervise également Kiruna --
missions polaires SSO
CTR-001,GS-TLS-01,2024-01-01,Paris HQ supervise la station de Toulouse
-- proximité géographique
CTR-002,GS-SGP-01,2023-03-15,Houston supervise Singapour -- couverture
zone Asie-Pacifique
CTR-003,GS-SGP-01,2024-02-01,Synchronisation depuis configuration
révisée
```

4) Script Index & EXPLAIN PLAN

Exercice 17 — Créer les index stratégiques manquants

17.A — Vérification des index déjà présents

```sql
SELECT
    index_name,
    table_name,
    uniqueness,
    visibility,
    status
FROM user_indexes
WHERE table_name IN ('SATELLITE', 'FENETRE_COM', 'PARTICIPATION')
ORDER BY table_name, index_name;
```

Résultat :

```
IDX_FENETRE_CODE_STATION,FENETRE_COM,NONUNIQUE,VISIBLE,VALID
IDX_FENETRE_DATETIME,FENETRE_COM,NONUNIQUE,VISIBLE,VALID
IDX_FENETRE_ID_SATELLITE,FENETRE_COM,NONUNIQUE,VISIBLE,VALID
PK_FENETRE_COM,FENETRE_COM,UNIQUE,VISIBLE,VALID
IDX_PART_ID_MISSION,PARTICIPATION,NONUNIQUE,VISIBLE,VALID
PK_PARTICIPATION,PARTICIPATION,UNIQUE,VISIBLE,VALID
SYS_IL0000073082C00004$$,PARTICIPATION,UNIQUE,VISIBLE,VALID
IDX_SATELLITE_ID_ORBITE,SATELLITE,NONUNIQUE,VISIBLE,VALID
IDX_SATELLITE_STATUT,SATELLITE,NONUNIQUE,VISIBLE,VALID
PK_SATELLITE,SATELLITE,UNIQUE,VISIBLE,VALID
```

```
UQ_SATELLITE_NOM,SATELLITE,UNIQUE,VISIBLE,VALID
```

Analyse

Cette requête permet de confirmer que les index fondamentaux des clés étrangères et des colonnes déjà identifiées en Phase 2 existent bien dans le schéma. Elle sert de point de départ avant l'ajout d'index complémentaires.

17.B — Création de l'index composite sur SATELLITE

```
CREATE INDEX IDX_SATELLITE_STATUT_ORBITE
ON SATELLITE (STATUT, ID_ORBITE);
```

Analyse

Cet index est utile pour les requêtes qui :

- filtrent d'abord les satellites par STATUT,
- puis les relient à leur orbite courante via ID_ORBITE.

Il améliore donc les traitements de reporting qui croisent l'état des satellites et leurs caractéristiques orbitales.

17.C — Création de l'index fonctionnel mensuel sur FENETRE_COM

```
CREATE INDEX IDX_FENETRE_MOIS
ON FENETRE_COM (TRUNC(DATETIME_DEBUT, 'MM'));
```

Analyse

Cet index fonctionnel est directement adapté aux besoins de la Phase 4, qui comporte de nombreuses requêtes de reporting mensuel, de cumul mensuel et de comparaison par mois. Il est particulièrement pertinent pour les agrégations par mois et les tableaux de bord.

17.D — Vérification finale après création

```
SELECT
    index_name,
    table_name,
    visibility,
    status
FROM user_indexes
WHERE table_name IN ('SATELLITE', 'FENETRE_COM', 'PARTICIPATION')
ORDER BY table_name, index_name;
```

Résultat :

```
IDX_FENETRE_CODE_STATION,FENETRE_COM,VISIBLE,VALID
IDX_FENETRE_DATETIME,FENETRE_COM,VISIBLE,VALID
IDX_FENETRE_ID_SATELLITE,FENETRE_COM,VISIBLE,VALID
IDX_FENETRE_MOIS,FENETRE_COM,VISIBLE,VALID
PK_FENETRE_COM,FENETRE_COM,VISIBLE,VALID
IDX_PART_ID_MISSION,PARTICIPATION,VISIBLE,VALID
PK_PARTICIPATION,PARTICIPATION,VISIBLE,VALID
SYS_IL0000073082C00004$$,PARTICIPATION,VISIBLE,VALID
IDX_SATELLITE_ID_ORBITE,SATELLITE,VISIBLE,VALID
IDX_SATELLITE_STATUT,SATELLITE,VISIBLE,VALID
IDX_SATELLITE_STATUT_ORBITE,SATELLITE,VISIBLE,VALID
PK_SATELLITE,SATELLITE,VISIBLE,VALID
UQ_SATELLITE_NOM,SATELLITE,VISIBLE,VALID
```

Analyse :

Les 2 nouveaux indexs sont bien présents.

Exercice 18 — Lire et interpréter un EXPLAIN PLAN

18.A — Requête de reporting mensuel

```sql
EXPLAIN PLAN FOR
SELECT
    TRUNC(f.datetime_debut, 'MM') AS mois,
    c.nom_centre,
    o.type_orbite,
    COUNT(f.id_fenetre) AS nb_fenetres,
    SUM(NVL(f.volume_donnees, 0)) AS volume_total
FROM FENETRE_COM f
JOIN STATION_SOL st
    ON f.code_station = st.code_station
JOIN AFFECTATION_STATION af
    ON st.code_station = af.code_station
JOIN CENTRE_CONTROLE c
    ON af.id_centre = c.id_centre
JOIN SATELLITE s
    ON f.id_satellite = s.id_satellite
JOIN ORBITE o
    ON s.id_orbite = o.id_orbite
WHERE f.statut = 'Réalisée'
GROUP BY
    TRUNC(f.datetime_debut, 'MM'),
```

```
    c.nom_centre,
    o.type_orbite;
```

18.B — Affichage du plan d’exécution

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Résultat :

```
PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------
--------
Plan hash value: 1113280566

---------------------------------------------------------------------------
--------
-------------------------------

| Id  | Operation                | Name                | Rows  |
Bytes | Cost (%CPU)| Time     |

---------------------------------------------------------------------------
--------
-------------------------------


PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------
--------
|   0 | SELECT STATEMENT              |                    |     3 |
  348 |    12  (17)| 00:00:01 |

|   1 |  HASH GROUP BY                |                    |     3 |
  348 |    12  (17)| 00:00:01 |

|   2 |   NESTED LOOPS                |                    |     3 |
  348 |    11  (10)| 00:00:01 |

|   3 |    NESTED LOOPS               |                    |     3 |
  348 |    11  (10)| 00:00:01 |

PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------
--------
```

```
|*  4 |     HASH JOIN                 |                    |     3 |
  312 |     8  (13)| 00:00:01 |

|   5 |      NESTED LOOPS             |                    |     3 |
  264 |     6  (17)| 00:00:01 |

|   6 |       NESTED LOOPS            |                    |     9 |
  264 |     6  (17)| 00:00:01 |

|   7 |        MERGE JOIN             |                    |     3 |

PLAN_TABLE_OUTPUT
---------------------------------------------------------------------
--------
  135 |     4  (25)| 00:00:01 |

|   8 |     TABLE ACCESS BY INDEX ROWID| CENTRE_CONTROLE          |     2
|
   54 |     2     (0)| 00:00:01 |

|   9 |      INDEX FULL SCAN     | PK_CENTRE_CONTROLE       |     2 |
      |     1     (0)| 00:00:01 |

|* 10 |     SORT JOIN            |                    |     3 |
   54 |     2  (50)| 00:00:01 |


PLAN_TABLE_OUTPUT
---------------------------------------------------------------------
--------
|  11 |      INDEX FULL SCAN     | PK_AFFECTATION_STATION   |     3 |
   54 |     1     (0)| 00:00:01 |

|* 12 |        INDEX RANGE SCAN        | IDX_FENETRE_CODE_STATION |     3
|
      |     0     (0)| 00:00:01 |

|* 13 |       TABLE ACCESS BY INDEX ROWID  | FENETRE_COM          |     1
|
   43 |     1     (0)| 00:00:01 |

|  14 |      VIEW                | index$_join$_008         |     5 |
   80 |     2     (0)| 00:00:01 |
```

```
PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------
--------

|* 15 |       HASH JOIN                |                    |       |
      |        |        |

|  16 |        INDEX FAST FULL SCAN    | IDX_SATELLITE_ID_ORBITE  |     5
|
   80 |     1     (0)| 00:00:01 |

|  17 |        INDEX FAST FULL SCAN    | PK_SATELLITE       |     5 |
   80 |     1     (0)| 00:00:01 |

|* 18 |     INDEX UNIQUE SCAN          | PK_ORBITE                |     1
|

PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------
--------
      |     0     (0)| 00:00:01 |

|  19 |    TABLE ACCESS BY INDEX ROWID      | ORBITE             |     1
|
   12 |     1     (0)| 00:00:01 |

---------------------------------------------------------------------------
--------
------------------------------


Predicate Information (identified by operation id):
---------------------------------------------------

PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------
--------

   4 - access("F"."ID_SATELLITE"="S"."ID_SATELLITE")
  10 - access("AF"."ID_CENTRE"="C"."ID_CENTRE")
       filter("AF"."ID_CENTRE"="C"."ID_CENTRE")
  12 - access("F"."CODE_STATION"="AF"."CODE_STATION")
  13 - filter("F"."STATUT"='Réalisée')
```

```
  15 - access(ROWID=ROWID)
  18 - access("S"."ID_ORBITE"="O"."ID_ORBITE")

Note
-----

PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------
--------
   - this is an adaptive plan
```

Analyse

Cette requête est représentative d'un véritable besoin de pilotage métier NanoOrbit. Elle mobilise :

FENETRE_COM pour les sessions de communication,
STATION_SOL et AFFECTATION_STATION pour rattacher les stations à leur centre,
CENTRE_CONTROLE pour l'organisation multi-sites,
SATELLITE et ORBITE pour la dimension technique et orbitale.

Dans le plan, il faut observer en priorité :

- la présence éventuelle de TABLE ACCESS FULL,
- l'utilisation éventuelle d'INDEX RANGE SCAN,
- le type de jointure choisi par Oracle.

Sur une petite base comme le jeu NanoOrbit, Oracle peut encore choisir des scans complets, car le coût estimé reste faible. Ce comportement n'est pas anormal. En revanche, sur un volume plus important, les index ajoutés à l'exercice 17 deviennent stratégiques pour limiter les accès complets.

18.C — Variante ciblée pour exploiter l'index fonctionnel

```
EXPLAIN PLAN FOR
SELECT
    id_fenetre,
    datetime_debut,
    volume_donnees
FROM FENETRE_COM
WHERE TRUNC(datetime_debut, 'MM') = DATE '2024-01-01'
ORDER BY datetime_debut;
```

Vérification :

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Résultat :

```
PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------
--------
Plan hash value: 1029258670

---------------------------------------------------------------------------
--------
--------------------

| Id  | Operation           | Name           | Rows  | Bytes | Cos
t (%CPU)| Time       |

---------------------------------------------------------------------------
--------
--------------------


PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------
--------
|   0 | SELECT STATEMENT          |                |   5 |   130 |
 2   (0)| 00:00:01 |

|   1 |  TABLE ACCESS BY INDEX ROWID| FENETRE_COM       |   5 |   130 |
 2   (0)| 00:00:01 |

|*  2 |   INDEX FULL SCAN         | IDX_FENETRE_DATETIME |   5 |     |
 1   (0)| 00:00:01 |

---------------------------------------------------------------------------
--------
--------------------

PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------
--------


Predicate Information (identified by operation id):
---------------------------------------------------

   2 -
```

```
filter(TRUNC(INTERNAL_FUNCTION("DATETIME_DEBUT"),'fmmm')=TO_DATE('
2024-0
1-01

            00:00:00', 'syyyy-mm-dd hh24:mi:ss'))

Note

PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------
--------
```

Analyse

Cette requête est intéressante car elle utilise directement l'expression indexée TRUNC(datetime_debut, 'MM'). Elle permet donc de vérifier si l'index IDX_FENETRE_MOIS peut être pris en compte par l'optimiseur.

Exercice 19 — Mesurer l'impact d'un index invisible

19.A — Vérification initiale de l'index

```
SELECT
index_name,
table_name,
visibility,
status
FROM user_indexes
WHERE index_name = 'IDX_SATELLITE_STATUT';
```

Résultat :

```
IDX_SATELLITE_STATUT,SATELLITE,VISIBLE,VALID
```

19.B — Rendre l'index invisible

```
ALTER INDEX IDX_SATELLITE_STATUT INVISIBLE;
```

Résultat :

```
IDX_SATELLITE_STATUT,SATELLITE,INVISIBLE,VALID
```

19.C — Relever le plan sans index visible

```sql
EXPLAIN PLAN FOR
SELECT
    id_satellite,
    nom_satellite,
    statut
FROM SATELLITE
WHERE statut = 'Opérationnel';
```

Résultat :

```
PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------
--------
Plan hash value: 2809638096

--------------------------------------------------------------------------
--------
-----------------------------------

| Id  | Operation                  | Name                | Rows
  | Bytes | Cost (%CPU)| Time    |

--------------------------------------------------------------------------
--------
-----------------------------------


PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------
--------
|   0 | SELECT STATEMENT                |                     |
3 |   114 | 2   (0)| 00:00:01 |

|   1 |  TABLE ACCESS BY INDEX ROWID BATCHED| SATELLITE           |
3 |   114 | 2   (0)| 00:00:01 |

|*  2 |   INDEX RANGE SCAN              | IDX_SATELLITE_STATUT_ORBITE |
3 |     |   1   (0)| 00:00:01 |

--------------------------------------------------------------------------
--------
-----------------------------------
```

```
PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------
--------


Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("STATUT"='Opérationnel')
```

Analyse

Lorsque l’index est invisible, Oracle ne peut plus le considérer dans son optimisation. Il peut alors privilégier un TABLE ACCESS FULL, en particulier sur un filtrage simple comme WHERE statut = 'Opérationnel'.

19.D — Rendre l’index visible à nouveau

```
ALTER INDEX IDX_SATELLITE_STATUT VISIBLE;
```

19.E — Relever le plan après réactivation

```
EXPLAIN PLAN FOR
SELECT
    id_satellite,
    nom_satellite,
    statut
FROM SATELLITE
WHERE statut = 'Opérationnel';
```

Résultat :

```
PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------
--------
Plan hash value: 3974464223

---------------------------------------------------------------------------
--------
----------------------------

| Id  | Operation                 | Name           | Rows  | Byt
es | Cost (%CPU)| Time     |
```

```
-----------------------------------------------------------------------
--------
----------------------------


PLAN_TABLE_OUTPUT
-----------------------------------------------------------------------
--------
|   0 | SELECT STATEMENT                |                |   3 |   1
14 |   2   (0)| 00:00:01 |

|   1 |  TABLE ACCESS BY INDEX ROWID BATCHED| SATELLITE       |   3 |
1
14 |   2   (0)| 00:00:01 |

|*  2 |   INDEX RANGE SCAN              | IDX_SATELLITE_STATUT |   3 |
   |   1   (0)| 00:00:01 |

-----------------------------------------------------------------------
--------
----------------------------

PLAN_TABLE_OUTPUT
-----------------------------------------------------------------------
--------


Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("STATUT"='Opérationnel')
```

Analyse

Quand l'index redevient visible, Oracle peut à nouveau choisir un plan exploitant un accès indexé. Il faut toutefois rester honnête dans l'analyse : sur une base de petite taille, l'optimiseur peut continuer à préférer un scan complet si son coût estimé reste plus faible. Ce résultat reste tout à fait acceptable dans le cadre d'un jeu de données pédagogique.

Rapport de pilotage intégral — Requête de synthèse finale

```
WITH volumes_centres AS (
    SELECT
        mois,
```

```
        nom_centre,
        SUM(volume_total) AS volume_centre
    FROM MV_VOLUMES_MENSUELS
    GROUP BY
        mois,
        nom_centre
),
classement AS (
    SELECT
        vc.mois,
        vc.nom_centre,
        vc.volume_centre,
        RANK() OVER (
            PARTITION BY vc.mois
            ORDER BY vc.volume_centre DESC
        ) AS rang_centre,
        ROUND(
            vc.volume_centre * 100 /
            NULLIF(SUM(vc.volume_centre) OVER (PARTITION BY vc.mois),
0),
            2
        ) AS part_volume_pct,
        LAG(vc.volume_centre) OVER (
            PARTITION BY vc.nom_centre
            ORDER BY vc.mois
        ) AS volume_mois_precedent
    FROM volumes_centres vc
),
satellites_rattaches AS (
    SELECT DISTINCT
        c.nom_centre,
        s.id_satellite,
        s.nom_satellite,
        s.statut
    FROM CENTRE_CONTROLE c
    JOIN AFFECTATION_STATION af
        ON c.id_centre = af.id_centre
    JOIN FENETRE_COM f
        ON af.code_station = f.code_station
    JOIN SATELLITE s
        ON f.id_satellite = s.id_satellite
)
SELECT
    cl.mois,
```

```
    cl.nom_centre,
    cl.rang_centre,
    cl.volume_centre,
    cl.part_volume_pct,
    cl.volume_mois_precedent,
    ROUND(
        CASE
            WHEN cl.volume_mois_precedent IS NULL OR
cl.volume_mois_precedent = 0
            THEN NULL
            ELSE ((cl.volume_centre - cl.volume_mois_precedent) /
cl.volume_mois_precedent) * 100
        END,
        2
    ) AS evolution_pct,
    sr.id_satellite,
    sr.nom_satellite,
    sr.statut
FROM classement cl
LEFT JOIN satellites_rattaches sr
    ON cl.nom_centre = sr.nom_centre
ORDER BY
    cl.mois,
    cl.rang_centre,
    sr.id_satellite;
```

Résultat :

```
2024-01-01,NanoOrbit Paris
HQ,1,3820,100,,,SAT-001,NanoOrbit-Alpha,Opérationnel
2024-01-01,NanoOrbit Paris
HQ,1,3820,100,,,SAT-002,NanoOrbit-Beta,Opérationnel
2024-01-01,NanoOrbit Paris
HQ,1,3820,100,,,SAT-003,NanoOrbit-Gamma,Opérationnel
```

Analyse du rapport final

Cette requête produit un tableau de bord opérationnel :

- elle agrège d'abord les volumes par centre et par mois,
- elle calcule ensuite le rang de chaque centre, sa part dans le volume total et son évolution par rapport au mois précédent grâce aux fonctions analytiques,
- enfin elle rattache les satellites concernés à chaque centre afin d'ajouter une lecture opérationnelle du statut de la flotte.

Cette logique est cohérente avec l'architecture NanoOrbit, où chaque centre de contrôle

supervise certaines stations tout en partageant un référentiel global de satellites et de missions.